

Report VIII

Lagrange polynomial interpolation

Author: Patryk Kuna

1. Introduction

Polynomial interpolation is a fundamental numerical method used mainly to approximate a function based on a finite set of known data points. Given a set of $(n+1)$ nodes (x_i, y_i) , it is possible to construct a polynomial of degree n such that it passes exactly through all these points.

One of the most commonly used forms is the Lagrange interpolation polynomial, defined as:

$$W_n(x) = \sum_{j=0}^n y_j \prod_{\substack{i=0 \\ i \neq j}}^n \frac{x - x_i}{x_j - x_i}$$

This formulation directly constructs the interpolating polynomial without solving a system of equations. However, its numerical behavior strongly depends on the distribution of interpolation nodes.

In this project, the interpolation quality is analyzed for the function:

$$y(x) = \frac{1}{1 + x^2}$$

Two types of nodes are considered:

- Equidistant nodes - uniformly distributed over the interval
- Chebyshev nodes - clustered near the boundaries to reduce oscillations

The goal is to compare their impact on interpolation accuracy and stability.

2. Description of the Numerical Task

The interpolation was performed in the interval:

$$x \in [-5, 5]$$

The following steps were carried out:

- generation of interpolation nodes
- computation of a function values as $y_i = y(x_i)$
- implementation of Lagrange polynomial
- evaluation of interpolation on a dense grid
- computation of the error
- visualization of results

Two scenarios were analyzed:

- Equidistant nodes
 - $n = 5$
 - $n = 10$
 - $n = 20$
- Chebyshev nodes
 - $n = 20$
 - $n = 50$

Additionally, interpolation error was evaluated as:

$$\varepsilon(x) = y(x) - W_n(x)$$

3. Numerical Method

The Lagrange interpolation polynomial is computed as a sum of weighted basis polynomials.

For each node j , the contribution is:

- numerator: product of $(x-x_i)$ for $i \neq j$
- denominator: product of (x_j-x_i) for $i \neq j$

The final value is obtained by summing:

$$W_n(x) = \sum y_j \cdot \frac{\alpha_j(x)}{\beta_j}$$

where:

- $\alpha_j(x)$ - product depending on evaluation point
- β_j - constant normalization factor

This method:

- exactly interpolates all nodes
- avoids solving linear systems
- may have the disadvantage from numerical instability for large n

4. Implementation

The program was written in C++ and followed closely the algorithm provided in the instruction file without using any numerical libraries.

Main steps of the implementation:

1. Data generation

- equidistant nodes:

$$x_i = x_{min} + i \cdot \Delta$$

- Chebyshev nodes:

$$x_i = \frac{x_{max} - x_{min}}{2} \cos\left(\frac{2i+1}{2n+2}\pi\right) + \frac{x_{min} + x_{max}}{2}$$

2. Function evaluation

- computation of $y(x)$

3. Lagrange polynomial

- nested loop implementation
- variables:
 - W - accumulator
 - alpha, beta - products

4. Data export

- interpolation values
- exact function
- interpolation error
- node positions

Data was saved to text files for plotting.

5. Results

The plots show the exact function, interpolation polynomial, and nodes. Exact function (red), interpolation (blue), nodes (points).

Case 1: Equidistant nodes, $n = 5$

- good general approximation
- noticeable deviation near boundaries
- low number of nodes limits accuracy

Case 2: Equidistant nodes, $n = 10$

- improved accuracy in central region
- oscillations begin near edges
- interpolation error increases toward boundaries

Case 3: Equidistant nodes, $n = 20$

- strong oscillations, a so called 'Runge phenomenon'
- very poor behavior near $x = \pm 5$
- large interpolation error

Case 4: Chebyshev nodes, $n = 20$

- significantly improved stability
- good approximation with the exact function
- reduced oscillations near the boundaries

Case 5: Chebyshev nodes, $n = 50$

- very high accuracy across entire interval
- perfect overlap with exact function
- error is minimal and evenly distributed

Figure 1. Interpolation for $n = 5$, equidistant nodes.

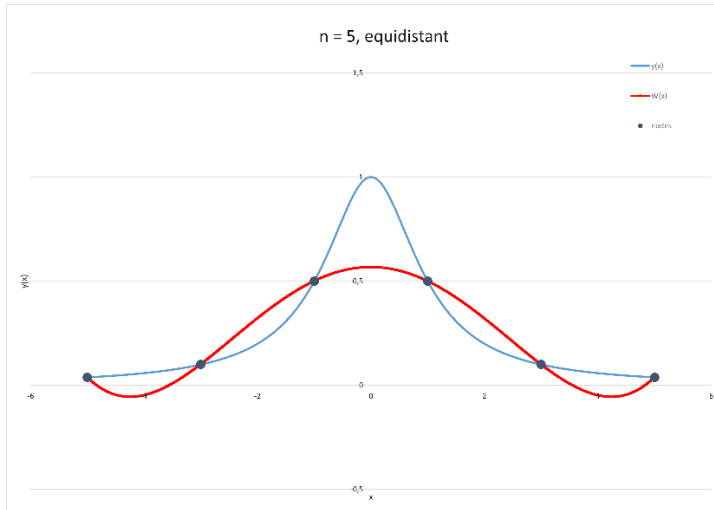


Figure 2. Interpolation for $n = 10$, equidistant nodes.

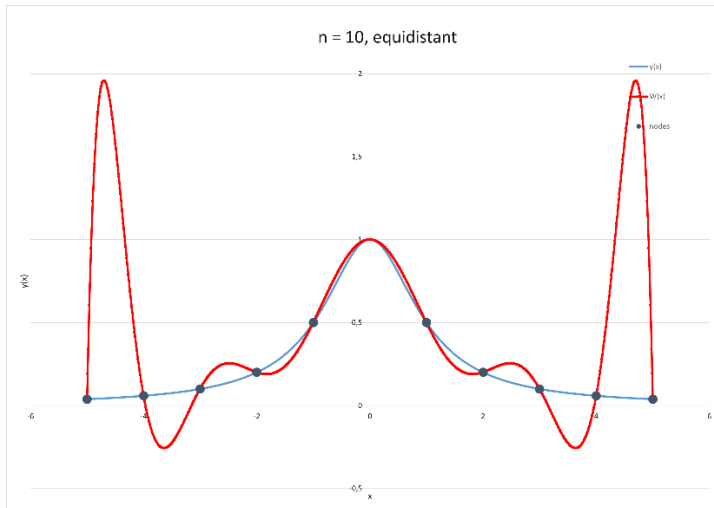


Figure 3. Interpolation for $n = 20$, equidistant nodes.

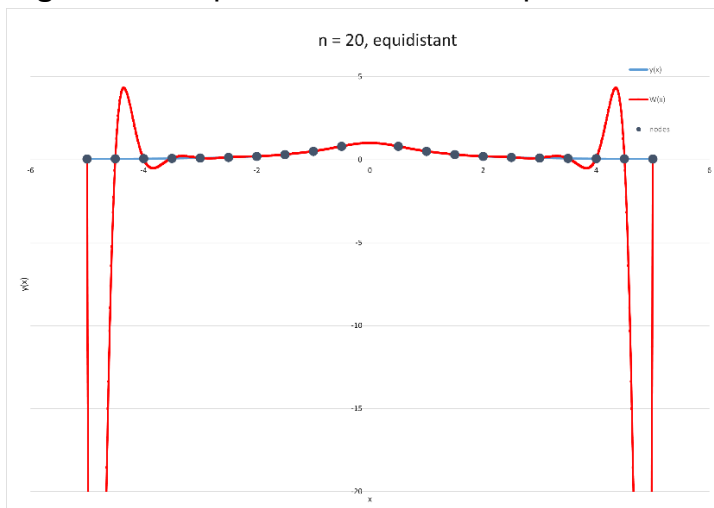


Figure 4. Interpolation for $n = 20$, Chebyshev nodes.

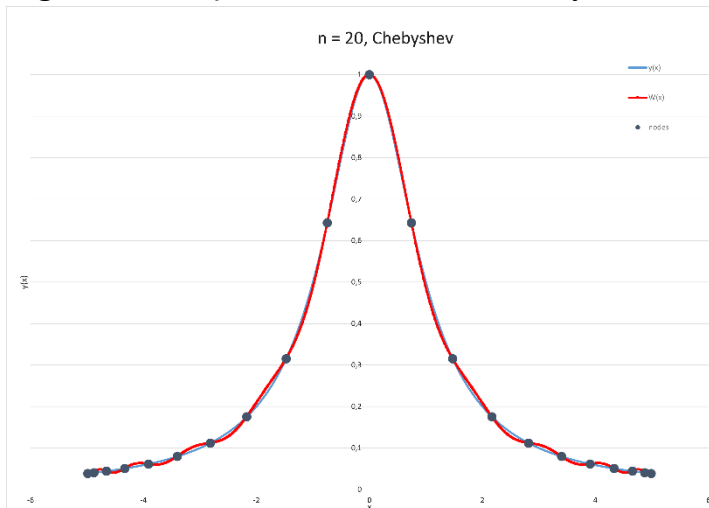
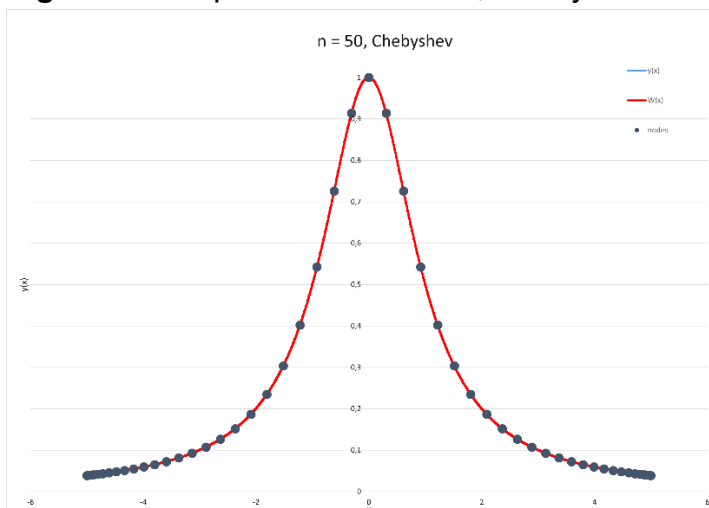


Figure 5. Interpolation for $n = 50$, Chebyshev nodes.



Interpolation Error for Chebyshev

- error remains really small across the domain
- no large spikes at the boundaries
- smooth oscillatory behavior

Figure 6. Interpolation error $\varepsilon(x)$ for $n = 20$.

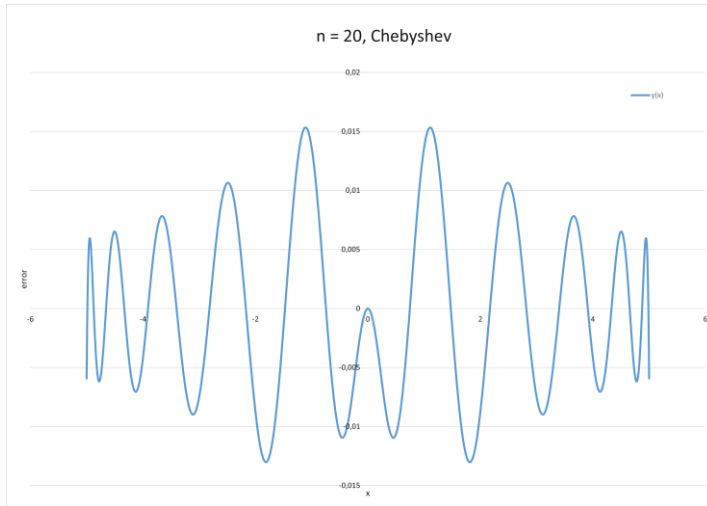
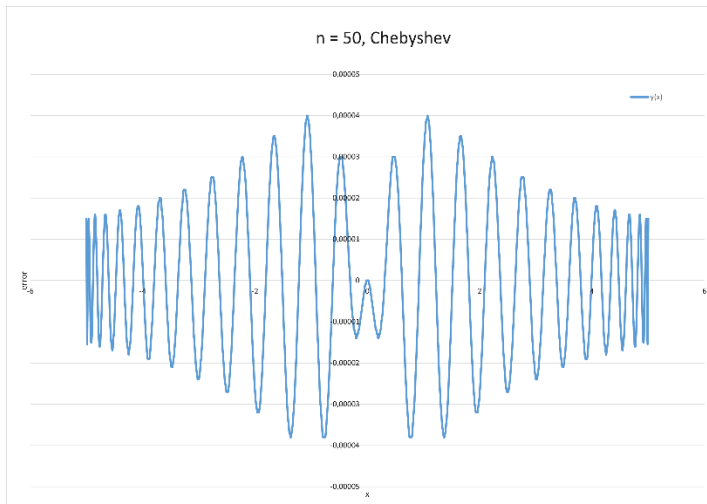


Figure 7. Interpolation error $\varepsilon(x)$ for $n = 50$.



6. Conclusions

The Lagrange interpolation method provides an exact fit at given nodes but its accuracy depends strongly on node distribution.

The following observations were made:

- increasing number of nodes improves the approximation only for proper node distributions
- equidistant nodes lead to strong oscillations for large n
- Chebyshev nodes significantly reduce interpolation error

- clustering nodes near boundaries improves stability
- interpolation error is much smaller and more uniform for Chebyshev nodes

Overall, the method is effective, but wise selection of nodes is crucial for obtaining accurate results.